INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DISCIPLINA: Padrões de Projeto em Aplicações Web
PROFESSOR: Éder Pereira
ATIVIDADE PRÁTICA: Padrão Comportamental Command

Contextualização do Problema

Estamos desenvolvendo um aplicativo para gerenciar playlists de música. Os usuários podem selecionar suas faixas favoritas e definir a ordem exata na qual devem ser executadas. Além de simplesmente reproduzir músicas em sequência, a playlist permite intercalar ações de controle de áudio, como aumentar ou diminuir o volume da execução.

Para a reprodução e controle de áudio, utilizamos uma biblioteca legada através da classe `Player`. Contudo, suas chamadas de método são síncronas e bloqueantes, e a execução direta acopla a interface do usuário às rotinas de áudio, impedindo funcionalidades como enfileiramento ou parametrização de ações.

Para solucionar esse problema, aplicaremos o padrão comportamental **Command**. O objetivo do **Command** é encapsular uma solicitação como um objeto, permitindo parametrizar clientes com diferentes requisições, enfileirar ou registrar solicitações e suportar operações que podem ser executadas sob demanda.

Roteiro de Implementação

1. **Criação do Projeto:**
    - Crie um projeto Java no Eclipse chamado `Command`.
2. **Criação da Classe do Recipiente (`Player`):**
    - Modele a classe `Player.java` que simula a biblioteca de áudio.

```java
public class Player {
    // Comportamento: Simular o tempo de reprodu  o de uma m sica
       exibindo mensagens de inicio, duracao e fim.
    public void play(String filename) throws InterruptedException {
        // TODO: Implementar a simula  o da execu  o do arquivo de
             udio  com Thread.sleep
    }

    // Comportamento: Exibir no console o n vel de aumento do volume.
    public void increaseVolume(int levels) {
        // TODO: Implementar o ajuste de aumento de volume
    }

    // Comportamento: Exibir no console o n vel de redu  o do volume.
    public void decreaseVolume(int levels) {
        // TODO: Implementar o ajuste de redu  o de volume
    }
}
```

3. **Definição da Interface do Comando (`Comando`):**
    - Crie a interface `Comando.java` para padronizar a execução de todas as ações.

```java
public interface Comando {
    void executa();
}
```

4. **Modelagem dos Comandos Concretos:**

   - Crie as classes `TocaMusicaComando.java`, `AumentaVolumeComando.java` e `DiminuiVolumeComando.java` implementando a interface `Comando`.
   - Defina em cada classe as referências necessárias para o `Player` e seus parâmetros (como nome do arquivo ou nível de volume).

```
public class TocaMusicaComando implements Comando {
    // TODO: Definir os atributos para o Player e para o arquivo de audio

    public TocaMusicaComando(Player player, String file) {
        // TODO: Inicializar os atributos via construtor
    }

    // Comportamento: Invocar o m todo play() do Player tratando
       poss veis exce  es.
    public void executa() {
        // TODO: Executar a chamada de reproducao do player
    }
}

public class AumentaVolumeComando implements Comando {
    // TODO: Definir os atributos para o Player e quantidade de n veis

    public AumentaVolumeComando(Player player, int levels) {
        // TODO: Inicializar os atributos via construtor
    }

    // Comportamento: Invocar o m todo increaseVolume() do Player.
    public void executa() {
        // TODO: Executar o aumento de volume
    }
}

public class DiminuiVolumeComando implements Comando {
    // TODO: Definir os atributos para o Player e quantidade de n veis

    public DiminuiVolumeComando(Player player, int levels) {
        // TODO: Inicializar os atributos via construtor
    }

    // Comportamento: Invocar o m todo decreaseVolume() do Player.
    public void executa() {
        // TODO: Executar a diminui  o de volume
    }
}
```

5. **Criação do Invocador/Enfileirador (`ListaDeComandos`):**

   - Crie a classe `ListaDeComandos.java` responsável por armazenar a sequência ordenada de comandos e executá-los em lote.

```
import java.util.ArrayList;
import java.util.List;

public class ListaDeComandos {
    // TODO: Definir a cole  o para armazenar os objetos do tipo Comando

    public ListaDeComandos() {
        // TODO: Inicializar a lista de comandos
```

```
    }

    // Comportamento: Adicionar um comando    fila de execu   o.
    public void adiciona(Comando comando) {
        // TODO: Implementar a inser   o na lista
    }

    // Comportamento: Iterar sobre a lista e executar cada comando
       sequencialmente.
    public void executa() {
        // TODO: Executar o m todo executa() de cada comando cadastrado
    }
}
```

6. **Testes do Sistema (`TestaListaDeComandos`):**

   - Desenvolva a classe de testes montando uma playlist personalizada composta por músicas e ajustes de áudio.

```
public class TestaListaDeComandos {
    public static void main(String[] args) {
        // TODO: Instanciar o Player e a ListaDeComandos
        // TODO: Adicionar sequencialmente comandos de reprodu   o e
           altera   es de volume na lista
        // TODO: Chamar o m todo executa() da ListaDeComandos para
           disparar a reprodu   o
    }
}
```